

Product Requirements Document (PRD)

Project: Iris Flower Classification using Machine Learning

Author: Hrithika

Type: Portfolio / Learning Project

Tools: Google Colab (development & training), VS Code (project structuring, scripts, GitHub prep)

Version: 1.0

Status: Planned

1. Overview

This project builds a supervised machine learning classifier that predicts the species of an Iris flower — Setosa, Versicolor, or Virginica — based on four physical measurements: sepal length, sepal width, petal length, and petal width. It uses the classic Iris dataset (built into Scikit-learn) and is intended as a foundational, portfolio-ready demonstration of the end-to-end ML classification workflow: data loading, exploration, preprocessing, model training, evaluation, and clean project packaging for GitHub.

2. Problem Statement

Given a set of flower measurements, can a model reliably determine which of three Iris species the flower belongs to? This is a classic multi-class classification problem, ideal for demonstrating core ML concepts (train/test splits, feature scaling, model comparison, evaluation metrics) without the overhead of data cleaning, since the dataset is already clean and well-structured.

3. Goals & Objectives

- Load and explore the Iris dataset using Scikit-learn.
- Visualize relationships between features and species.
- Train at least one classification model (with a comparison of 2–3 algorithms recommended).
- Evaluate performance using accuracy, precision, recall, F1-score, and a confusion matrix.
- Package the project in a clean, GitHub-ready repository structure.
- Produce a well-documented notebook (Colab) and a modular script version (VS Code) so the project is usable both as a learning notebook and as a reusable pipeline.

4. Non-Goals

- No deployment (API/web app) is in scope for this version — noted as a future extension only.

- No web-scraped or custom-collected dataset — this project uses the Kaggle "Iris.csv" dataset (a standard, pre-cleaned version of the classic Iris dataset).
- No deep learning / neural network approach — classical ML algorithms only (this is a foundational project).

5. Target Audience

- Recruiters/reviewers assessing entry-level Data Analyst / Data Scientist / AI-ML Engineer portfolios.
- Personal learning reference for classification fundamentals.

6. Dataset

Attribute	Detail
Source	Kaggle — Iris.csv by saurabh00007 (a standard, pre-cleaned mirror of the classic UCI Iris dataset)
File	Iris.csv, downloaded manually and stored in data/
Samples	150 (50 per class)
Columns	Id, SepalLengthCm, SepalWidthCm, PetalLengthCm, PetalWidthCm, Species
Target classes	Iris-setosa, Iris-versicolor, Iris-virginica
Missing values	None (clean dataset)
Note	The Id column is not a feature and must be dropped before training; Species is the target label and needs encoding (e.g. LabelEncoder) before model training

7. Tech Stack

Layer	Tool/Library
Language	Python 3.x
Data handling	pandas, numpy
Visualization	matplotlib, seaborn
ML modeling	scikit-learn (Logistic Regression, KNN, Decision Tree, SVM — pick/compare)
Model persistence	joblib or pickle
Development	Google Colab (exploration + training)
Code structuring	VS Code (scripts, modularization, Git)
Version control	Git & GitHub

8. Functional Requirements

1. **Data Loading** — Load Iris dataset into a pandas DataFrame with proper column names and species labels.
2. **Exploratory Data Analysis (EDA)** — Summary statistics; pair plots / scatter plots by species; correlation heatmap.
3. **Preprocessing** — Train/test split (e.g., 80/20, stratified by class); feature scaling (StandardScaler) where the algorithm requires it.
4. **Model Training** — Train at least one classifier; ideally compare Logistic Regression, KNN, and Decision Tree/SVM.
5. **Evaluation** — Accuracy score, confusion matrix, classification report (precision/recall/F1), and a simple cross-validation check.
6. **Model Saving** — Persist the trained model (.pkl/.joblib) so it can be reloaded without retraining.
7. **Documentation** — A clear README explaining the problem, approach, results, and how to run the project.

9. Success Metrics

- Model achieves $\geq 95\%$ **accuracy** on the test set (Iris is a well-separated dataset, so this is a realistic bar).
- Clear, reproducible results with visual confusion matrix and metrics table.
- Clean, working GitHub repository that runs end-to-end without errors when cloned fresh.

10. Project Structure (GitHub-ready)

```
iris-flower-classification/
|
|-- README.md
|-- requirements.txt
|-- .gitignore
|-- LICENSE
|
|-- data/
|   |-- Iris.csv                # dataset downloaded from Kaggle (saurabh00007/iris.csv)
|
|-- notebooks/
|   |-- iris_classification.ipynb  # main Colab notebook (EDA + training + evaluation)
|
|-- src/
|   |-- __init__.py
|   |-- data_loader.py          # loads & prepares the dataset
|   |-- preprocess.py           # train/test split, scaling
|   |-- train.py                # model training logic
|   |-- evaluate.py             # metrics, confusion matrix, reports
|   |-- predict.py              # load saved model & run predictions
|
|-- models/
|   |-- iris_model.pkl          # saved trained model
|
|-- images/
|   |-- pairplot.png
```

```
|   |-- confusion_matrix.png
|
|-- tests/
|   |-- test_model.py           # basic sanity/unit tests
```

Why this structure:

- notebooks/ keeps your exploratory Colab work separate from production-style code — recruiters can see your thinking process in the notebook.
- src/ holds modular, reusable Python scripts (this is what you'd write/refine in VS Code) — shows you can go beyond notebooks.
- models/ and images/ store generated artifacts so the repo is self-contained and results are visible without re-running code.
- requirements.txt + README.md make the repo instantly runnable and reviewable by anyone (including recruiters).

11. Project Build Structure (Step-by-Step Pipeline)

This is the actual sequence of work — what you build, in what order, and why each step feeds the next. This is different from the folder layout (section 10); this is the **process**.

```
STEP 1: Load Data
  pd.read_csv("data/Iris.csv") -> pandas DataFrame
  Drop "Id" column
  |
  v
STEP 2: Explore Data (EDA)
  .describe(), pairplot, correlation heatmap
  -> Understand which features separate species best
  |
  v
STEP 3: Preprocess
  Train/test split (80/20, stratified)
  Feature scaling (StandardScaler) if needed
  |
  v
STEP 4: Train Model(s)
  Logistic Regression / KNN / Decision Tree / SVM
  Fit on training data
  |
  v
STEP 5: Evaluate
  Predict on test data
  Accuracy, confusion matrix, precision/recall/F1
  -> Pick the best-performing model
  |
  v
STEP 6: Save Model
  joblib.dump(model, "iris_model.pkl")
  |
  v
STEP 7: Reload & Predict
  joblib.load("iris_model.pkl") -> predict on new input
  |
  v
```

STEP 8: Package & Push to GitHub
 README + requirements.txt + folder structure + commit + push

Each step maps to one section of your notebook and, later, one script file in `src/`. That mapping is what makes the "Colab-to-VS-Code" transition below easy instead of confusing.

12. How Colab and VS Code Connect (This Is the Part People Get Stuck On)

Colab and VS Code are **not linked live** — there's no auto-sync. You move code and files between them manually, in one direction: **Colab is where you experiment, VS Code is where you clean up and finalize.** Here's exactly how that works:

Stage A — Get the dataset into Colab, then work there first

Download `Iris.csv` from [Kaggle](#) and upload it into your Colab session (either drag-and-drop into the Colab file panel each session, or upload it once to Google Drive and mount the Drive so it persists). Then write and run all your code in a single Colab notebook (`iris_classification.ipynb`), doing Steps 1–7 from the pipeline above, top to bottom, in notebook cells. You are debugging and exploring here — messy code is fine at this stage.

Stage B — Export the files Colab produces

By the end of Stage A, you have:

- 1. **The dataset itself** — `Iris.csv` (the same file you uploaded — this is what goes into the GitHub repo's `data/` folder)
- 2. **The trained model file** — `iris_model.pkl` (created via `joblib.dump()`)
- 3. **Plot images** — e.g. `pairplot.png`, `confusion_matrix.png` (created via `plt.savefig()`)

Download these from Colab's file panel (left sidebar → files → download), or mount Google Drive and save them there, then pull them onto your laptop.

Stage C — Move to VS Code and split the notebook into scripts

This is the actual "connection": you take the working code from your Colab cells and copy each logical block into its own `.py` file inside `src/`, matching the pipeline steps:

Colab notebook cell(s) did this...	...becomes this VS Code file
Step 1 (load data)	<code>src/data_loader.py</code>
Step 3 (split + scale)	<code>src/preprocess.py</code>
Step 4 (train model)	<code>src/train.py</code>
Step 5 (evaluate)	<code>src/evaluate.py</code>
Step 7 (reload + predict)	<code>src/predict.py</code>

Each script becomes a small, reusable function instead of a notebook cell. For example, code that was a loose cell in Colab:

```
df = pd.read_csv("data/Iris.csv")
df = df.drop(columns=["Id"])
```

becomes a proper function in `src/data_loader.py`:

```
def load_iris_data(path="data/Iris.csv"):
    import pandas as pd
    df = pd.read_csv(path)
    df = df.drop(columns=["Id"])
    return df
```

Stage D — Place the exported files into the project folder

The `Iris.csv` goes into `data/`. The `iris_model.pkl` you downloaded from Colab goes into `models/`. The `.png` plots go into `images/`. Now your local VS Code project folder is fully self-contained — it doesn't need Colab to run anymore; `predict.py` can load `models/iris_model.pkl` directly with `joblib.load()`, and `train.py` can re-read `data/Iris.csv` directly with `pd.read_csv()`.

Stage E — Run and verify locally in VS Code

In the VS Code terminal:

```
pip install -r requirements.txt
python src/train.py          # re-trains and re-saves the model locally, confirming it matches Colab's result
python src/predict.py       # loads the saved model and runs a sample prediction
```

If this runs cleanly with no Colab dependency, your project is portable — which is exactly what GitHub reviewers will test when they clone your repo.

Stage F — Push to GitHub

Once verified locally, `git add`, `git commit`, `git push` from VS Code's terminal or Source Control panel. The notebook (`notebooks/`) and the scripts (`src/`) both go into the repo — the notebook shows your thought process, the scripts show you can productionize it.

One-line summary of the connection: Colab is the lab, VS Code is where the lab results get organized into a real, runnable project — the model file and images are the physical link between them; the notebook-to-script mapping is the code link.

13. README Contents (recommended)

- Project title & one-line description
- Problem statement
- Dataset description
- Approach/methodology summary
- Results (accuracy, confusion matrix image, metrics table)
- How to run (setup + commands)
- Tech stack badges
- Future improvements section

14. Milestones / Timeline

Milestone	Deliverable
1. Setup & EDA	Dataset loaded, visualized, insights noted (Colab)
2. Preprocessing	Train/test split + scaling implemented
3. Model training	2–3 models trained and compared
4. Evaluation	Metrics, confusion matrix, best model selected
5. Modularization	Code refactored into <code>src/</code> scripts (VS Code)
6. Packaging	README, requirements.txt, folder structure finalized
7. GitHub push	Repository live, tested via fresh clone

15. Future Enhancements (out of scope for v1)

- Deploy as a simple Streamlit/Flask app for interactive predictions.
- Add hyperparameter tuning (GridSearchCV).
- Extend to a REST API endpoint.
- Add CI (GitHub Actions) to auto-run tests on push.

16. Risks / Considerations

- Iris is a small, well-separated dataset — very high accuracy is expected and should not be over-stated as a sign of general ML skill; the README should frame this as a fundamentals project, not a complex real-world case.
- Keep the notebook and scripts in sync — avoid the common issue of a repo where the notebook and `src/` code diverge.
- Credit the dataset source in the README (link to the [Kaggle dataset page](#)) since it's a third-party download, not one generated by your own code.
- The Kaggle CSV uses `Iris-setosa / Iris-versicolor / Iris-virginica` as string labels — remember to encode these (e.g. `LabelEncoder`) before feeding them into most Scikit-learn classifiers.